Politecnico di Torino

Politecnico di Torino

Computer Engineering (AI & Data Analytics)

A.a. 2025/2026

Graduation Session September 2026

Generative AI and Foundation Models for Automated Data Engineering and Metadata Orchestration

Enhancing Data Pipelines with AI

Supervisor: Paolo Garza

Candidate: Marc'Antonio Lopez

Acknowledgements

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodo sed egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

Table of Contents

List of Figures

Chapter 1

Introduction

1.1 Context and Motivation

Modern enterprises accumulate thousands of database tables across operational systems, data warehouses, and analytics platforms. Each table carries a schema that encodes business logic, yet the documentation describing that logic—requirement specifications, glossaries, data policies—rarely stays synchronized with the evolving database. This *semantic gap* between business documentation and physical data infrastructure is a core challenge in data governance. Regulatory frameworks such as GDPR impose strict requirements on data lineage, metadata accuracy, and auditability, making the problem not merely technical but also organizational [1].

Data stewards address this gap through manual cataloging, lineage tracking, and ad-hoc querying of system architecture. The process is labor-intensive, error-prone, and does not scale to the volume and velocity of changes typical of enterprise environments. Business documentation speaks in domain terms—"revenue," "customer churn," "subscription plan"—while database schemas use technical identifiers such as `TBL_REVENUE_FY`, `CUST_CHRN_FLG`, and `SUB_PLAN_CD`. Bridging this vocabulary automatically would transform data governance from a reactive, manual task into a proactive, systematic capability.

Large Language Models (LLMs)—built on the Transformer architecture [2]—have demonstrated strong capabilities in text understanding, information extraction, and structured reasoning [3]. Retrieval-Augmented Generation (RAG) has emerged as a practical paradigm for grounding LLM outputs in real data [4]. However, baseline RAG—which retrieves flat document chunks via vector similarity—struggles with global sensemaking over entire document corpora and fails to capture the relational structure inherent in data governance domains [5].

Graph-based RAG (GraphRAG) represents an evolution of this paradigm: by structuring retrieved knowledge as graphs rather than flat embeddings, it enables

richer, interconnected retrieval that preserves entity relationships and supports hierarchical reasoning [6, 7]. This thesis leverages GraphRAG principles, combined with multi-stage entity resolution and self-reflective LLM orchestration, to address the specific challenges of automated data governance.

1.2 Problem Statement

This thesis addresses the following problem: given a collection of heterogeneous governance documents—requirement specifications, data glossaries, DDL scripts, and architectural diagrams—and an enterprise database schema, automatically construct a knowledge graph that maps business concepts to physical tables and enables natural-language querying of the resulting system architecture.

Existing approaches fall short along several dimensions. Traditional data catalogs (e.g., Collibra, Alation) require extensive manual configuration and offer limited natural-language capabilities. Baseline RAG systems handle factual lookup but fail on cross-document reasoning and complex schema relationships. Microsoft GraphRAG [5] and related systems [8] address graph-based reasoning but are not designed for data governance specifically—they lack entity resolution across heterogeneous sources, schema mapping validation, and incremental update mechanisms.

The specific challenges addressed in this thesis include:

- **Semantic gap:** bridging business terminology and technical schema identifiers across documents with different structures and vocabularies.
- **Entity deduplication:** identifying and merging entities that refer to the same business concept despite surface-level differences in naming, language, or context [9].
- **Mapping validation:** ensuring that the relationships between business concepts and physical database tables are semantically correct.
- **Query correctness:** generating accurate queries against the constructed knowledge graph; hallucination is mitigated through self-reflective critique mechanisms [10].
- **Incremental updates:** supporting partial reprocessing when documents change, without reconstructing the entire knowledge graph.

1.3 Contributions

This thesis makes the following contributions:

1. **Two-graph architecture for data governance.** A strict separation between a *Builder* pipeline—which constructs the knowledge graph from governance documents—and a *Query* pipeline—which answers natural-language questions over the graph. This separation mirrors CQRS (Command Query Responsibility Segregation) principles applied to AI pipelines, enabling independent execution and incremental updates.

2. **Multi-stage entity resolution.** A two-stage approach combining embedding-based vector blocking (Union-Find with cosine similarity clustering) with an LLM-based judge for precise deduplication. This hybrid approach is language-agnostic and applicable across heterogeneous governance documents.

3. **Self-reflection loops for quality assurance.** Three independent correction mechanisms: (i) Actor-Critic validation for schema mapping, extending the iterative generate-feedback-refine loop of Self-Refine [11] to use separate Actor and Critic LLM instances (a design choice specific to this thesis, as Self-Refine uses a single model for all roles); (ii) Cypher healing for query generation, where generated queries are validated via `EXPLAIN` and errors are fed back for correction; and (iii) hallucination grading for answer quality, following Self-RAG principles [10].

4. **Hybrid retrieval with ablation-backed design.** A retrieval strategy combining dense vector search, BM25 keyword matching, and graph traversal with reciprocal rank fusion. The design is validated through systematic ablation studies across multiple synthetic datasets.

5. **Provider-agnostic LLM factory.** A five-tier architecture supporting multiple LLM providers (OpenRouter, OpenAI, Anthropic, local models) with automatic provider detection and fallback, enabling cost-performance optimization across pipeline stages.

6. **Incremental update mechanism.** A hash-based change detection system that identifies modified document chunks and triggers partial reprocessing, avoiding full graph reconstruction when documents change.

7. **Evaluation framework with custom AI Judge.** A systematic ablation study evaluated through a domain-specific AI Judge rather than generic frameworks such as RAGAS [12]. While RAGAS employs LLM-based judges for its core metrics, its *answer relevancy* metric relies on cosine similarity between generated and reference question embeddings—a mechanism that penalizes technical responses containing database identifiers and domain-specific jargon, where semantically equivalent questions diverge in embedding space. Empirical testing confirmed this mismatch: fully grounded answers consistently scored

below 0.2 on answer relevancy. Additionally, RAGAS's *context precision* penalizes the large parent chunks required for graph-structured retrieval, and its metrics cannot reward correct negative answers (i.e., "I don't know"). A custom LLM-as-a-Judge [13] prompt is therefore designed to be architecture-aware—explicitly encoding the system's pipeline structure, known limitations, and domain constraints to provide more meaningful evaluation across multiple weighted dimensions.

1.4 Thesis Structure

The remainder of this thesis is organized as follows:

- **Chapter 2** provides the technical background and surveys related work, covering natural language processing foundations, retrieval-augmented generation, graph-based RAG systems, LLM orchestration patterns, and evaluation methodologies for generative systems.
- **Chapter 3** presents the high-level solution design, including the two-graph architecture, functional requirements, and the overall system design at an architectural level.
- **Chapter ??** details the implementation, covering the LangGraph-based orchestration, data models, prompt engineering strategies, and the individual pipeline components.
- **Chapter ??** describes the experimental evaluation, including the synthetic dataset generation, the AI Judge methodology, ablation study design, and results analysis.
- **Chapter ??** summarizes the findings, discusses limitations, and outlines directions for future work.

Chapter 2

Related Work and Background

This chapter surveys the technical foundations and related work that underpin the contributions of this thesis. We begin with the fundamentals of natural language processing, proceed through retrieval-augmented generation and its graph-based evolution, cover LLM orchestration and reasoning patterns, and conclude with evaluation methodologies for generative systems.

2.1 Foundations of Natural Language Processing

2.1.1 Text Representation and Embeddings

Representing text as numerical vectors is a prerequisite for any computational language task. Early approaches used one-hot encodings and bag-of-words representations, which are sparse and fail to capture semantic similarity. The introduction of distributed representations—Word2Vec [14] and GloVe [15]—marked a fundamental shift: words are mapped to dense vectors in a continuous space where geometric proximity reflects semantic relatedness.

Static embeddings assign a single vector per word regardless of context, conflating distinct senses of polysemous words. Contextual embeddings address this limitation by producing word representations that depend on the surrounding sentence. ELMo (Embeddings from Language Models) [16] pioneered this approach using bidirectional LSTMs, while BERT [17] advanced it through masked language modeling over Transformer encoders, producing representations that capture deep bidirectional context.

For retrieval tasks, sentence-level and document-level embeddings are essential. Contrastive learning approaches train encoders to produce similar embeddings

for semantically related text pairs while pushing unrelated pairs apart. The E5 family [18] extends this paradigm to multilingual settings through weakly-supervised contrastive pre-training on large-scale multilingual text pairs followed by supervised fine-tuning. BGE-M3 [19], used in this thesis, further supports dense, sparse, and multi-vector representations within a single model through multi-granularity objectives, enabling flexible retrieval strategies.

The quality of embeddings directly impacts downstream retrieval performance. Cosine similarity between embedding vectors serves as the primary distance metric for semantic search, and embedding dimension, training data diversity, and contrastive loss design all influence retrieval effectiveness in practice. Figure 2.1 illustrates the evolution from sparse representations to dense contextual embeddings.

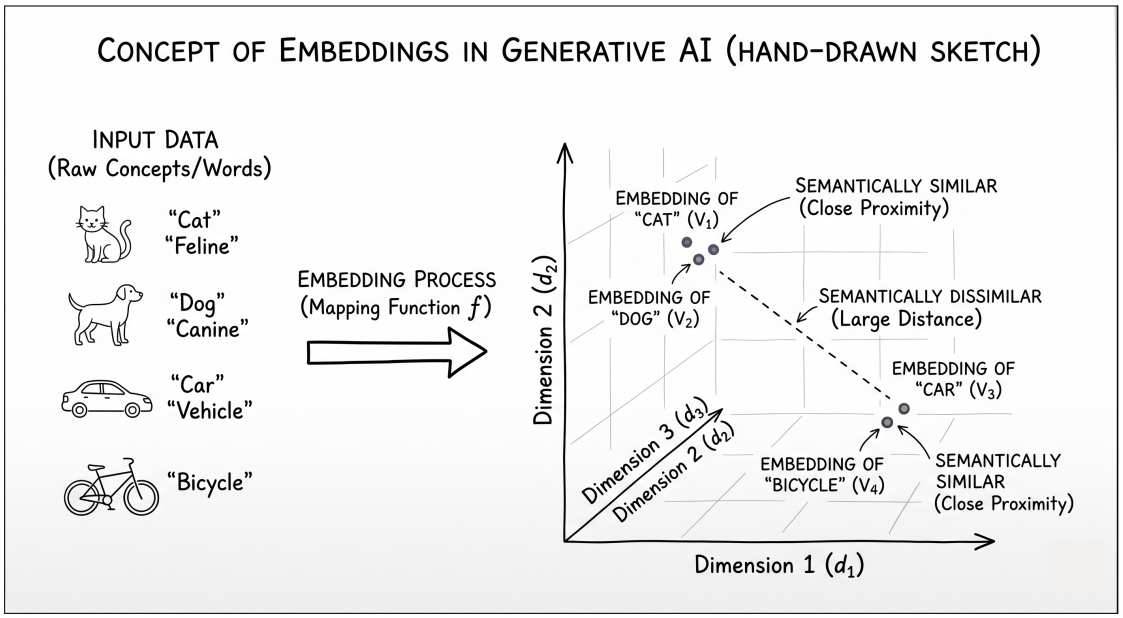

Figure 2.1: Dense representations capture semantic similarity in continuous vector spaces.

2.1.2 Transformer Architecture and Large Language Models

The Transformer architecture [2] replaced recurrent and convolutional layers with self-attention mechanisms that compute pairwise interactions between all positions in a sequence. Multi-head attention allows the model to attend to different representation subspaces simultaneously, while positional encodings inject sequence

order information. The architecture's inherent parallelism enables efficient training on large corpora. Figure 2.2 illustrates the core Transformer architecture and its three main variants.

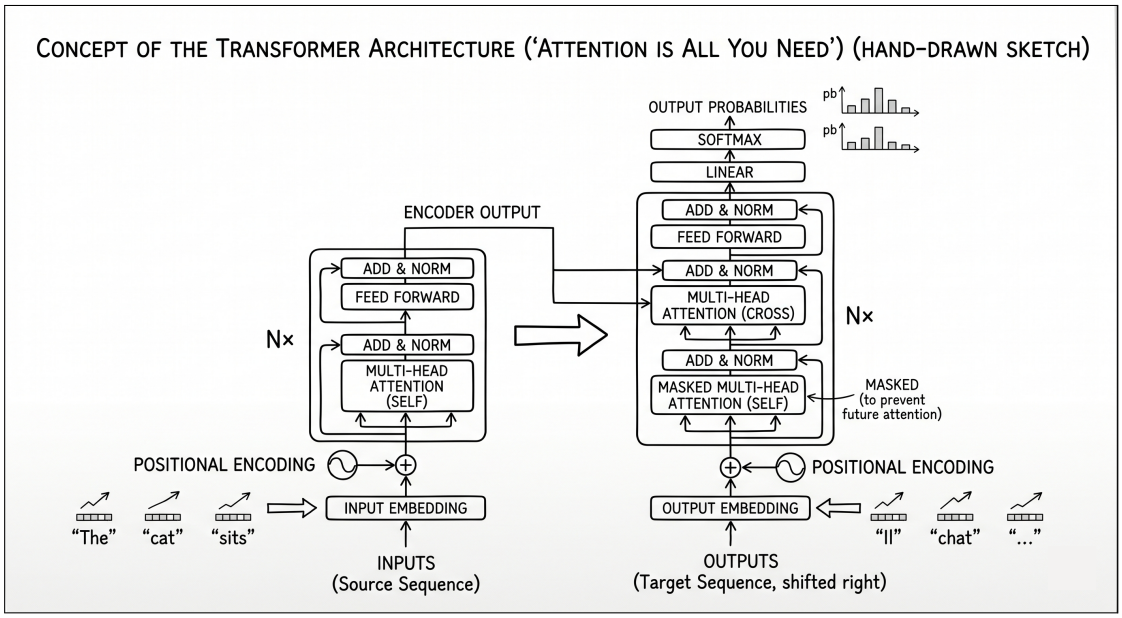

Figure 2.2: The original Transformer architecture and its three variants: encoder-only (BERT), decoder-only (GPT), and encoder-decoder (T5). Each variant is optimized for different task families.

Three main Transformer variants have emerged. *Encoder-only* models (BERT [17]) process tokens bidirectionally and excel at classification, entity recognition, and embedding tasks. *Decoder-only* models (GPT-3 [3] and its successors) generate text autoregressively and form the backbone of modern LLMs. *Encoder-decoder* models (T5, BART) combine both directions for sequence-to-sequence tasks such as translation and summarization.

Scale has proven transformative. As models grow beyond billions of parameters, new capabilities emerge that are absent in smaller models—a phenomenon documented through chain-of-thought prompting [20], where large models generate explicit reasoning chains that dramatically improve performance on arithmetic, commonsense, and symbolic reasoning tasks. Post-training alignment techniques—RLHF (Reinforcement Learning from Human Feedback) [21] and instruction tuning—further refine model behavior, transforming raw next-token predictors into instruction-following agents.

The LLM landscape in 2024–2026 spans commercial APIs (OpenAI, Anthropic, Google), open-weight models (Llama [22], Mistral [23]), and local deployment options. This diversity motivates the provider-agnostic architecture proposed in

this thesis, which abstracts LLM access behind a unified factory interface.

2.2 Retrieval-Augmented Generation

2.2.1 Baseline RAG Pipeline

The standard RAG pipeline [4] operates in two phases. During *ingestion*, documents are split into chunks, embedded into vector representations, and stored in a vector index. During *query*, the user question is embedded, the top-k most similar chunks are retrieved via approximate nearest-neighbor search, and the retrieved chunks are assembled into a prompt context for the LLM to generate an answer. Figure 2.3 illustrates the baseline RAG pipeline.

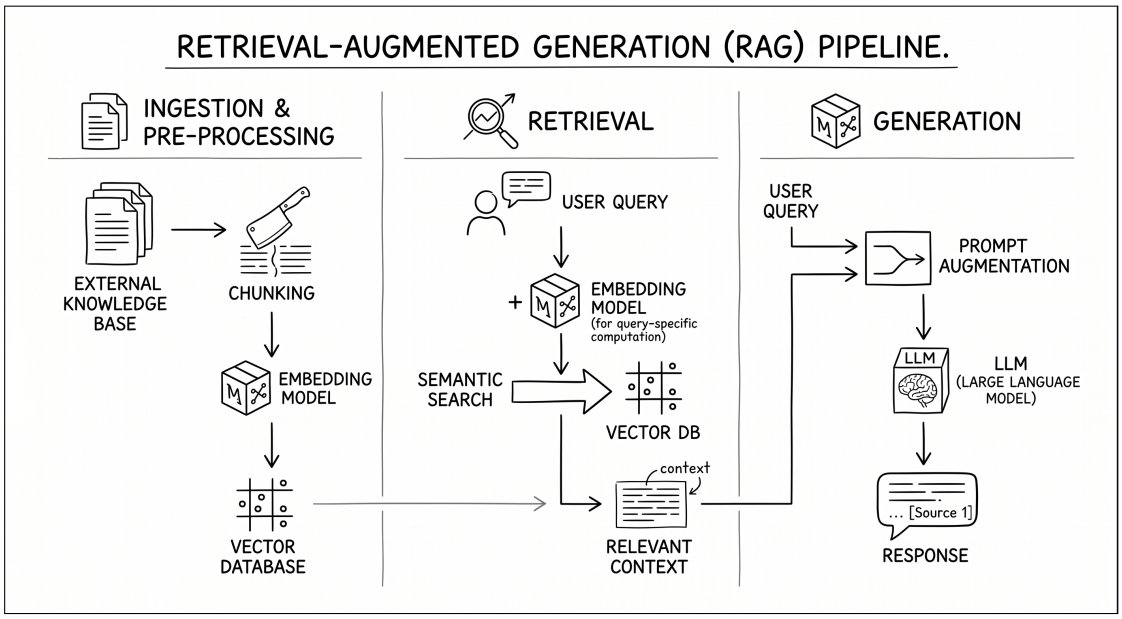

Figure 2.3: The baseline RAG pipeline: documents are chunked and embedded during ingestion; at query time, the top-k chunks are retrieved and provided as context to the LLM.

Chunking strategy significantly affects retrieval quality. Fixed-size chunking with overlap is the simplest approach, but fails to respect document structure. Semantic chunking segments text at topic boundaries, while hierarchical segmentation [24] constructs multi-level representations by clustering segments into higher-level units. A systematic investigation by Shaukat et al. [25] demonstrates that chunking strategy and embedding model choice interact in non-trivial ways, with no single strategy dominating across all tasks.

Retrieval can be further refined through reranking. A common pattern uses a bi-encoder for efficient candidate generation followed by a cross-encoder for precise relevance scoring. Déjean et al. [26] provide a thorough comparison showing that cross-encoders remain competitive with LLM-based rerankers for the computational cost. Figure 2.4 illustrates the bi-encoder vs. cross-encoder paradigm. The granularity of retrieval units also matters: Chen et al. [27] show that proposition-level retrieval—where the unit is a distinct factual claim—outperforms passage-level and sentence-level retrieval in terms of generalization across diverse tasks, although it faces challenges with multi-hop reasoning over long-range text.

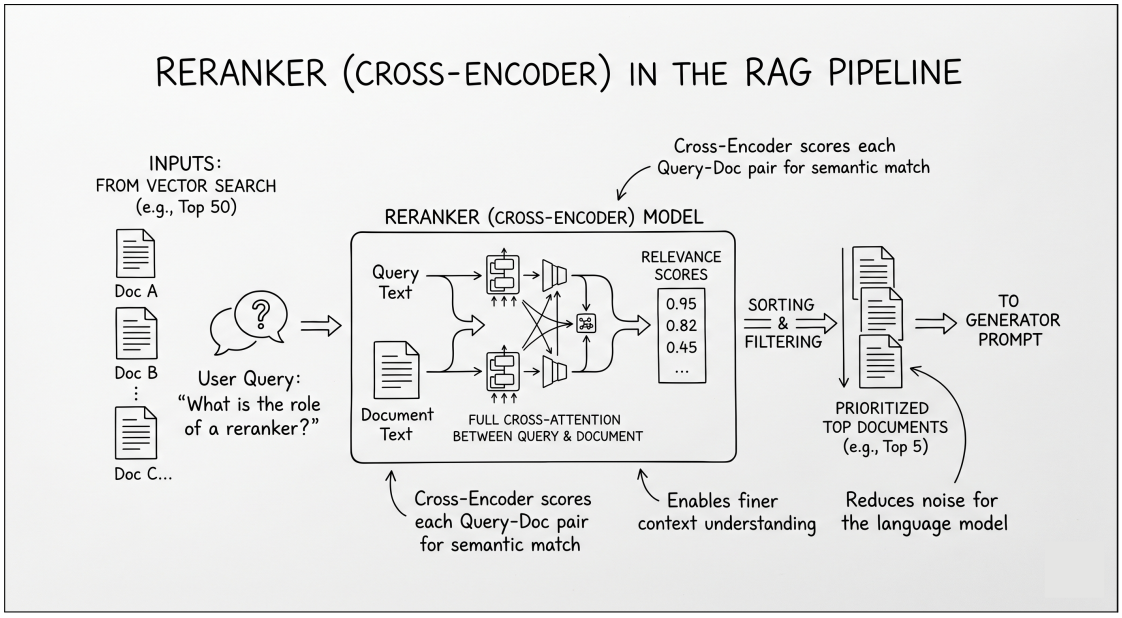

Figure 2.4: While the bi-encoder encodes query and document independently for fast retrieval, the cross-encoder processes query-document pairs jointly for precise relevance scoring.

2.2.2 Corrective and Self-Reflective RAG

Baseline RAG treats retrieval as a single, unassessed step: retrieved chunks are passed to the generator regardless of relevance. This assumption fails when the retrieval is noisy, incomplete, or misleading. Two families of corrective mechanisms address this limitation.

Corrective RAG (CRAG) [28] introduces a lightweight retrieval evaluator that assesses the confidence of retrieved documents. When confidence is low—indicating that retrieved chunks are likely irrelevant—CRAG triggers corrective actions such

as web search or query refinement before generation. This defensive pattern significantly improves robustness on out-of-distribution queries.

Self-RAG [10] goes further by enabling the model itself to control the retrieval-generation process through reflection tokens. The model predicts a `<retrieve>` token with values `yes/no/continue` that gate on-demand retrieval, followed by critique tokens: `ISREL` ({relevant, irrelevant}), `ISSUP` ({fully supported, partially supported, no support}), and `ISUSE` (a 1–5 utility scale)—to self-assess generation quality and trigger regeneration when outputs lack grounding. This creates a self-reflective loop where the model critiques its own outputs and iteratively improves them. Figure 2.5 compares the CRAG and Self-RAG corrective mechanisms.

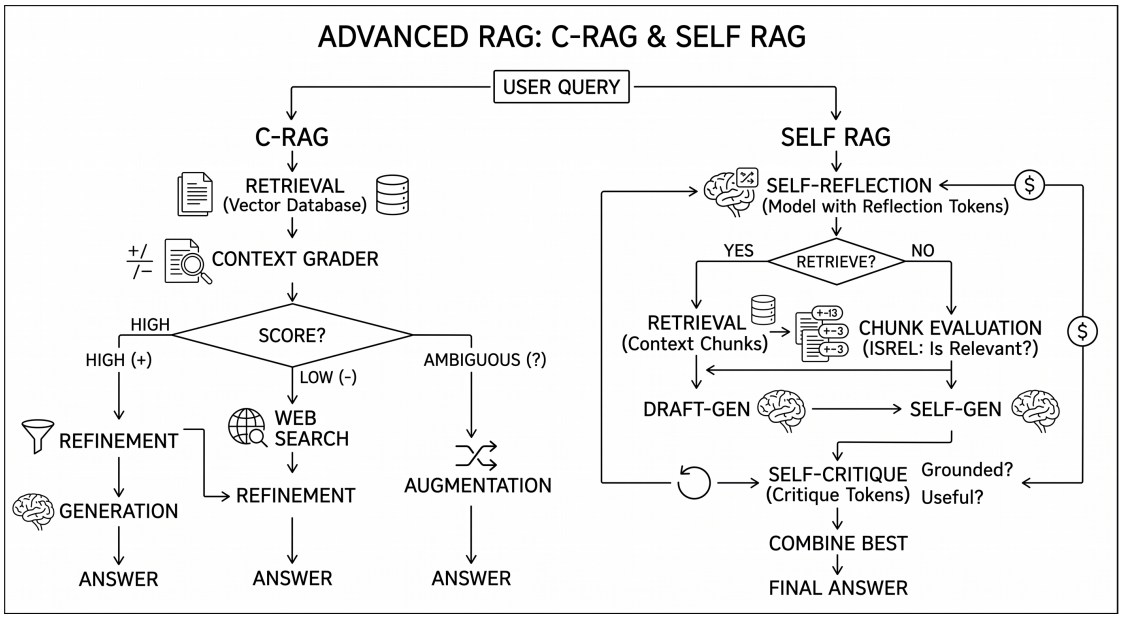

Figure 2.5: Corrective RAG (left) introduces a retrieval evaluator that triggers corrective actions on low confidence; Self-RAG (right) enables the model to emit reflection tokens that control retrieval and self-assess generation quality.

In this thesis, the hallucination grading mechanism in the Query pipeline implements a Self-RAG-inspired pattern: a grader LLM evaluates the groundedness of generated answers against the retrieved context and triggers regeneration cycles when hallucinations are detected.

2.2.3 Agentic RAG

The evolution from single-pass retrieval to autonomous agents represents the current frontier of RAG systems. Agentic RAG [29] replaces the fixed retrieve-then-generate pipeline with an agent that plans retrieval strategies, uses tools (search engines,

SQL queries, APIs), and iteratively refines its answers based on intermediate observations.

The foundational pattern is ReAct [30], which interleaves *Thought* (reasoning about the current state), *Action* (executing a tool call), and *Observation* (processing the result) in a loop until the task is complete. ReAct demonstrates competitive performance with chain-of-thought reasoning on multi-step benchmarks such as HotpotQA and FEVER, with particular improvements in reducing hallucination through grounding in real observations rather than relying solely on parametric knowledge. Figure 2.6 illustrates the evolution from baseline RAG to agentic RAG with the ReAct loop.

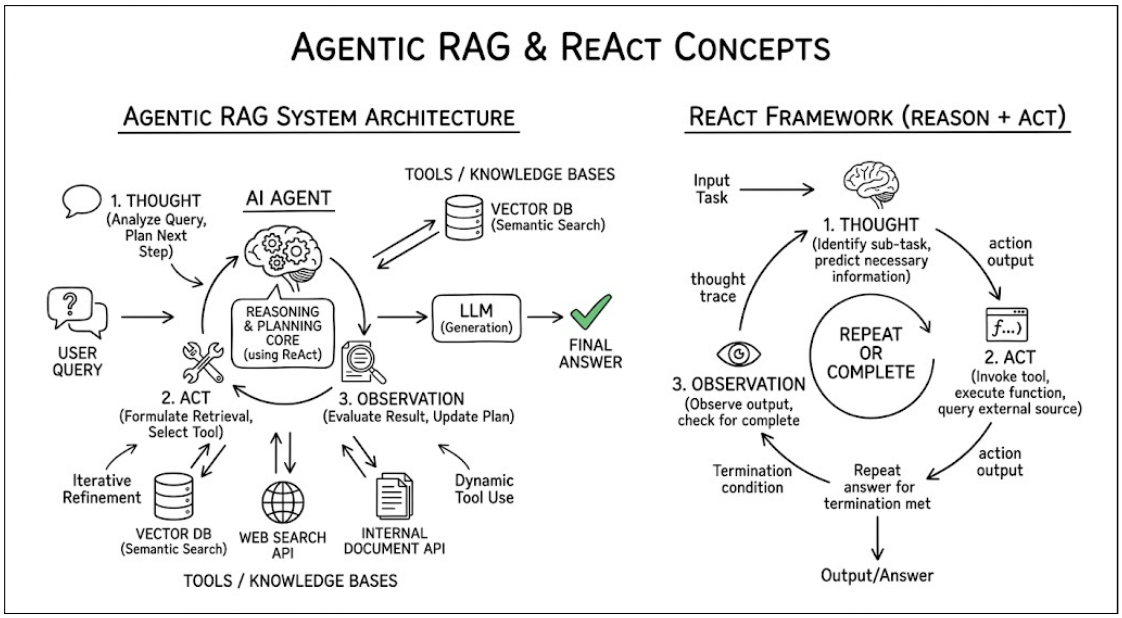

Figure 2.6: Evolution from baseline RAG (single retrieve-then-generate) to agentic RAG: the ReAct loop interleaves Thought, Action, and Observation, enabling iterative refinement with external tools.

A survey by Wang et al. [31] identifies four core modules of LLM-based autonomous agents: *profiling* (defining the agent's role and capabilities), *memory* (short-term context and long-term knowledge), *planning* (decomposing tasks into executable steps), and *action* (interacting with external tools and environments). These modules provide a useful taxonomy for understanding agentic systems.

The Query pipeline in this thesis implements a multi-step agentic RAG architecture with conditional routing, quality gates, and self-correction mechanisms, orchestrated via LangGraph state machines that define the agent's behavior graph.

2.3 Graph-Based Retrieval-Augmented Generation

2.3.1 Knowledge Graph Construction with LLMs

Knowledge graphs represent information as networks of entities (nodes) connected by typed relationships (edges), with optional properties on both nodes and edges. Traditional construction methods—rule-based extraction, manual curation, crowdsourcing—struggle with scalability and adaptability to new domains.

LLMs offer a compelling alternative: prompted with appropriate instructions, they can extract structured entity-relation triplets (subject, predicate, object) from unstructured text, achieving competitive triplet yields compared to specialized pretrained models—though careful prompt engineering is required to ensure entity normalization and relation quality [32]. The extraction process typically involves: (i) segmenting documents into processing units, (ii) prompting the LLM to identify entities and their relationships, (iii) structuring the output as triplets (often enforced via JSON-mode outputs), and (iv) merging extracted triplets into a unified graph.

Neo4j serves as a natural backend for knowledge graph storage. Its property graph model, Cypher query language, and support for vector indexes make it suitable for both graph traversal and semantic search within the same database. The `MERGE` operation in Cypher provides idempotent writes—ensuring that reprocessing does not create duplicate nodes or edges—which is essential for incremental update scenarios.

2.3.2 GraphRAG Systems

GraphRAG extends the RAG paradigm by structuring external knowledge as knowledge graphs rather than flat document collections. Two comprehensive surveys [6, 7] provide taxonomies of graph-based RAG methods, identifying key dimensions such as graph construction strategy, retrieval mechanism and granularity, and generation approach.

Microsoft GraphRAG [5] represents the foundational system in this space. Its pipeline processes documents through: (i) text unit extraction, (ii) entity and relationship extraction via LLMs, (iii) Leiden community detection for hierarchical clustering, (iv) community-level summarization, and (v) construction of a graph index over the hierarchical community structure. The paper describes a *Global Search* query mode that aggregates community-level summaries via map-reduce to answer broad questions; the open-source implementation further extends this with *Local Search* (entity-centric retrieval) and *DRIFT* (cross-community reasoning).

GRAG [8] focuses on preserving graph topology during retrieval, arguing that the structural relationships between entities carry information that flat retrieval

discards. By operating on subgraphs rather than individual nodes, GRAG maintains relational context that improves structural reasoning.

Security considerations are also relevant: Liang et al. [33] demonstrate that GraphRAG systems are vulnerable to knowledge graph poisoning attacks, where adversarial text injected into the source corpus is processed into manipulated entities that distort downstream answers. This motivates robust validation mechanisms in the construction pipeline. Figure 2.7 illustrates the GraphRAG paradigm.

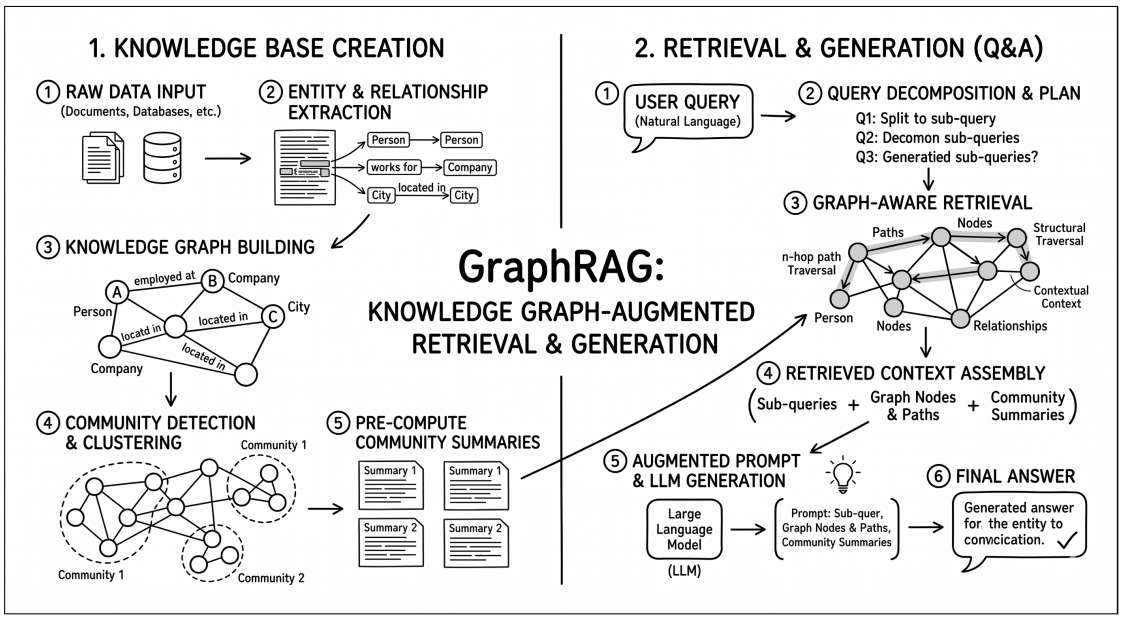

Figure 2.7: The GraphRAG paradigm: documents are processed into a knowledge graph via LLM-based extraction; queries traverse the graph structure to retrieve interconnected subgraphs for answer generation.

Table 2.1 compares the key characteristics of major GraphRAG systems [5, 34, 8].

2.3.3 Entity Resolution in Knowledge Graphs

Entity resolution—identifying and merging records that refer to the same real-world entity—is critical for knowledge graph quality. In governance domains, the same business concept may appear across documents with different names ("Customer" vs. "Client" vs. "CLI"), in different languages, or with different levels of specificity.

Traditional approaches use rule-based matching and probabilistic record linkage, which require domain-specific configuration and struggle with semantic variability [9]. Dense vector representations offer an alternative path: by encoding entities

Table 2.1: Comparison of GraphRAG systems.

Feature	**MS GraphRAG**	**LightRAG**	**GRAG**	**SemanticMesh**
Graph construction	LLM-based	LLM-based	LLM-based	LLM-based
Community detection	Leiden	None	None	Cosine clustering
Entity resolution	Limited	None	None	Embedding + LLM
Schema mapping	No	No	No	Yes
Self-correction	No	No	No	Multi-loop
Incremental updates	Partial	No	No	Hash-based
Domain-specific	General	General	General	Data governance

as embeddings, candidate pairs can be identified through similarity search regardless of surface-level differences.

Oliya et al. [35] propose an end-to-end differentiable approach that unifies entity resolution and question answering, demonstrating that resolution quality directly impacts downstream QA performance. Their insight—that resolution should be integrated into the query pipeline rather than treated as a separate preprocessing step—informs the design of the Builder pipeline in this thesis.

The hybrid approach adopted here combines embedding-based blocking (using Union-Find with cosine similarity thresholds to cluster candidate duplicates) with LLM-based adjudication (where a judge model evaluates each cluster and decides whether to merge). This two-stage design balances recall (the blocking step captures candidates that rule-based methods would miss) with precision (the LLM judge provides semantic understanding that pure similarity scores cannot).

2.4 LLM Orchestration and Agentic Reasoning

2.4.1 Prompt Engineering Techniques

Prompt engineering—the design of input instructions to steer LLM behavior—has emerged as a discipline in its own right. The Prompt Report [36] surveys 58 text-only prompting techniques organized into six major categories: zero-shot, few-shot, thought generation, decomposition, ensembling, and self-criticism.

Several techniques are directly relevant to this thesis. *Few-shot prompting* provides in-context examples that guide output format and behavior without modifying model weights. *Role prompting* defines the LLM's persona and task framing, establishing expectations about output style and domain expertise. *Output format specification*—particularly JSON-mode enforcement—ensures that extraction tasks produce structured, parseable results rather than free-form text.

Table 2.2 illustrates how these techniques compose into a single prompt for entity extraction—a task central to the Builder pipeline. The prompt combines role assignment, output format specification, and a few-shot example to constrain the LLM into producing structured, parseable results.

The shift from fine-tuning to prompt engineering is significant: this thesis uses no fine-tuning whatsoever, relying entirely on carefully designed prompts and orchestration logic. This choice reflects the practical reality that fine-tuning requires curated training data, significant compute, and ongoing maintenance—whereas prompt engineering can adapt to new domains and requirements without retraining.

Table 2.2: Composed prompt example for entity extraction, combining role, few-shot, and output format techniques.

Component	Content
Role	You are a data governance analyst. You extract business entities from requirement documents and map them to database concepts.
Task	Extract all business entities from the following document excerpt. For each entity, provide the entity name, a short description, and the most likely database table it maps to.
Output Format	Respond strictly in JSON. Each entity must have keys: `name`, `description`, `mapped_table`.
Few-shot Example	`{ "name": "Customer", "description": "An individual or organization that purchases subscriptions", "mapped_table": "CUSTOMER_MASTER" }`
Input	*[document excerpt]*

2.4.2 Reasoning and Acting Patterns

Chain-of-Thought (CoT) prompting [20] demonstrated that providing LLMs with reasoning exemplars elicits explicit reasoning chains that dramatically improve performance on arithmetic, commonsense, and symbolic reasoning tasks. Kojima et al. [37] further showed that even zero-shot CoT—simply appending "Let's think step by step"—improves reasoning in large models. On the GSM8K math benchmark, few-shot CoT prompting improved accuracy from 17.9% to 56.9%. Figure 2.8 illustrates the chain-of-thought prompting paradigm.

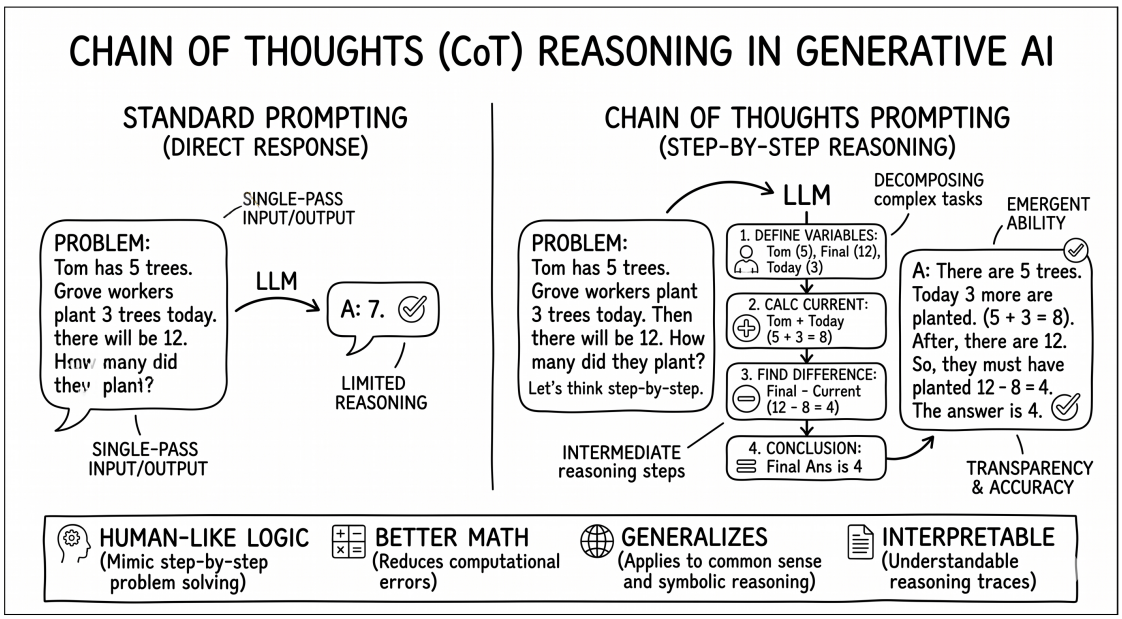

Figure 2.8: Chain-of-Thought prompting: standard prompting (left) produces direct answers, while CoT prompting (right) elicits intermediate reasoning steps that improve accuracy on complex tasks.

ReAct [30] extends CoT by interleaving reasoning with action. Rather than reasoning purely in the abstract, ReAct agents execute tool calls (search, lookup, calculation) and incorporate the results into their reasoning. This grounding in real observations reduces hallucination and produces interpretable traces that reveal the agent's decision-making process. On benchmarks such as FEVER, ReAct outperforms both pure CoT and pure acting approaches.

The LangGraph-based orchestration in this thesis implements ReAct-style patterns at the pipeline level: each graph node performs an action (extraction, mapping, validation, generation), and state transitions between nodes encode the reasoning flow. Conditional edges enable dynamic routing based on intermediate results, implementing the "thought" component of the ReAct loop.

2.4.3 Self-Reflection and Self-Correction

Self-reflection mechanisms enable LLMs to improve their own outputs through iterative feedback, without external human intervention or gradient updates.

Reflexion [38] introduces verbal reinforcement learning: after an agent completes a task, it generates a natural-language reflection on its performance, which is stored in an episodic memory buffer. On subsequent attempts, the agent retrieves relevant reflections to avoid repeating mistakes. Multi-trial experiments show consistent improvement across reasoning and coding tasks.

Self-Refine [11] uses a single LLM in a generate-feedback-refine loop: the model produces an initial output, generates feedback on its own output, and then refines the output based on the feedback. No external model or training data is required. The approach is validated across seven diverse tasks including dialogue generation, code optimization, and mathematical reasoning.

The Actor-Critic validation in this thesis adapts the Self-Refine generate-feedback-refine loop by assigning separate Actor and Critic roles to different LLM instances: an Actor LLM proposes a schema mapping, a Critic LLM evaluates it, and the critique is fed back as a reflection prompt for the next iteration. Similarly, the Cypher healing mechanism—where generated queries are validated via `EXPLAIN` and errors trigger regeneration—implements a self-correction loop grounded in the database's own query planner.

Related work on Text-to-Cypher generation provides additional context. Auto-Cypher [39] uses an LLM-supervised pipeline for generating synthetic Cypher query data with built-in quality assurance through iterative execution and validation, showing improvements over single-pass generation approaches. SyntheT2C [40] addresses the data scarcity problem for Text-to-Cypher by generating synthetic training data. This thesis takes a different approach: rather than fine-tuning on synthetic data, it uses few-shot prompting with carefully selected examples, combined with the self-correction loop described above.

2.5 Evaluation Methodologies for RAG Systems

2.5.1 Automated Evaluation with LLM-as-Judge

Evaluating generative systems is inherently challenging: outputs are open-ended, multiple valid answers may exist, and human evaluation is expensive and difficult to scale. The LLM-as-Judge paradigm [13] addresses this by using a strong LLM to evaluate the outputs of other LLMs.

Zheng et al. demonstrate that GPT-4 as a judge achieves over 80% agreement with human experts on MT-Bench, a multi-turn benchmark—comparable to the agreement rate between human annotators themselves. However, identified

biases include position bias (preferring the first of two options), verbosity bias (favoring longer outputs), and self-enhancement bias (a tendency to favor outputs generated by the same model), although the authors note that the evidence for self-enhancement bias remains inconclusive.

These findings motivate the design of the evaluation framework in this thesis: rather than pairwise comparison, the AI Judge uses an absolute scoring rubric with five weighted dimensions, reducing susceptibility to pairwise biases while maintaining the scalability advantages of automated evaluation.

2.5.2 RAG Evaluation Frameworks

RAGAS [12] provides reference-free metrics for RAG evaluation across four core dimensions: *faithfulness* (is the answer grounded in the context?), *answer relevancy* (does the answer address the question?), *context precision* (are retrieved chunks relevant?), and *context recall* (is all necessary information retrieved?). Faithfulness, context precision, and context recall employ LLM-based judges to assess groundedness and relevance. Answer relevancy follows a hybrid approach: the LLM generates hypothetical questions from the answer, and these are compared to the original question via cosine similarity over embeddings.

While RAGAS is well-suited for evaluating factual question-answering over documents, it proved inadequate for the governance use case in this thesis for three concrete reasons. First, the answer relevancy metric's reliance on embedding cosine similarity systematically penalizes technical responses containing database identifiers, table names, and domain-specific jargon—semantically equivalent questions diverge in embedding space when the vocabulary is highly specialized. Empirical testing confirmed this: fully grounded answers consistently scored below 0.2 on answer relevancy. Second, context precision penalizes the large parent chunks (800+ tokens) required by graph-structured retrieval, as the LLM judge tends to issue "partially relevant" verdicts for chunks that contain relevant information alongside broader context. Third, RAGAS metrics cannot reward correct negative answers: when the system correctly responds "I don't know," both context recall and faithfulness default to zero, penalizing appropriate uncertainty.

The alternative adopted in this thesis is a custom AI Judge that is explicitly aware of the governance domain architecture. The judge receives the system's architectural context alongside each evaluation sample, enabling it to assess answer quality against domain-specific criteria. This approach draws from the LLM-as-Judge methodology but adapts it to the specific requirements of data governance evaluation, as detailed in Chapter **??**.

2.6 Summary and Positioning

This chapter has surveyed the technical landscape relevant to this thesis, spanning NLP foundations, RAG paradigms, GraphRAG systems, LLM orchestration patterns, and evaluation methodologies. Table 2.3 positions this thesis—the SemanticMesh system—relative to existing approaches.

Table 2.3: Feature comparison of related systems versus SemanticMesh (this thesis).

Capability	Data Catalogs	MS GraphRAG	GRAG	Light-RAG	Semantic-Mesh
Automated KG construction	Partial	Yes	Yes	Yes	Yes
Entity resolution	Manual	Limited	No	No	Yes
Schema mapping	Manual	No	No	No	Yes
Self-correction loops	No	No	No	No	Yes
Incremental updates	No	Partial	No	No	Yes
Domain-specific design	No	No	No	No	Yes
Ablation-based evaluation	No	No	No	No	Yes

The key gap identified in the literature is that no existing system combines (i) automated knowledge graph construction from heterogeneous governance documents, (ii) validated schema mapping between business concepts and physical tables, (iii) multi-loop self-correction mechanisms, (iv) incremental update support, and (v) systematic ablation-based evaluation. The SemanticMesh system addresses this gap by integrating and extending techniques from the GraphRAG, entity resolution, self-reflection, and agentic RAG literature into a unified framework specifically designed for data governance automation.

Chapter 3

Solution Design

This chapter presents the high-level design of SemanticMesh, a multi-agent framework for Data Governance automation that bridges the semantic gap between unstructured business documentation and relational database schemas by autonomously constructing a Knowledge Graph. We first analyse the problem and derive the system requirements (Section 3.1), then describe the two-graph architecture that separates ontology construction from query answering (Section 3.2), detail the core design patterns that enable self-correction and robustness (Section 3.3), and conclude with a discussion of the key design decisions and their trade-offs (Section 3.4).

3.1 Problem Analysis and Requirements

3.1.1 Problem Formulation

As discussed in Chapter 1, data stewards in enterprise environments must manually align business semantics—expressed in unstructured glossaries, data dictionaries, and policy documents—with physical database schemas defined through DDL statements. This alignment, known as semantic mapping, is a prerequisite for regulatory compliance [1], data lineage tracking, and effective data cataloguing. The manual process is inherently slow, subjective, inconsistent across stewards, and does not scale to schemas with hundreds of tables.

Formally, the problem can be stated as follows. Given a set of unstructured documents $\mathcal{D} = \{d_1, \ldots, d_n\}$ (PDF business glossaries, data dictionaries) and a set of relational schemas $\mathcal{S} = \{s_1, \ldots, s_m\}$ expressed as DDL statements, construct a Knowledge Graph $\mathcal{G} = (V, E)$ where the nodes V include both business concepts V_B extracted from $\mathcal{D}$ and physical table representations V_T parsed from $\mathcal{S}$, and the edges E include semantic mappings $E_M \subseteq V_B \times V_T$ that align each business

concept to its corresponding physical implementation.

This formulation raises five specific challenges:

- **Semantic gap.** Business documentation uses natural language (e.g., "net revenue"), while database schemas use abbreviated identifiers (e.g., `TBL_REV_FY`). Bridging this gap requires both linguistic understanding and domain knowledge.
- **Entity deduplication.** The same business concept may appear under different surface forms across documents ("Customer", "client", "CLI"). Extracted entities must be resolved to a single canonical representation.
- **Mapping validation.** Automatically generated mappings must be validated for semantic correctness, not merely syntactic well-formedness, before being committed to the Knowledge Graph.
- **Query correctness.** Natural-language questions over the Knowledge Graph must be answered using only information grounded in the source documents, avoiding hallucinated claims.
- **Incremental updates.** When source documents or schemas change, the Knowledge Graph must be updated incrementally rather than rebuilt from scratch.

3.1.2 Functional Requirements

From the problem analysis, we derive the following functional requirements:

1. **End-to-end semantic mapping automation.** The system shall accept PDF documents and DDL files as input, autonomously extract business concepts, parse physical schemas, propose semantic mappings, validate them, and persist the results as a Knowledge Graph.
2. **Multi-database and multi-schema support.** The system shall handle DDL schemas from different databases simultaneously, mapping concepts across heterogeneous schemas with distinct naming conventions.
3. **Full traceability (provenance).** Every business concept and mapping in the Knowledge Graph shall carry a reference to the source text fragment from which it was derived, enabling audit and verification.
4. **Natural-language question answering.** The system shall accept natural-language queries over the constructed Knowledge Graph and return grounded answers with explicit source references.

5. **Human-in-the-loop override.** For low-confidence mappings that fall below a configurable threshold, the system shall pause and present the proposal for human review before committing it to the Knowledge Graph.

3.1.3 Non-Functional Requirements

Table 3.1 summarises the non-functional requirements that constrain the system design. These requirements directly motivate several architectural choices discussed in subsequent sections.

Table 3.1: Non-functional requirements guiding the system design.

ID	Requirement	Category
NFR-01	Re-running the Builder on the same input produces identical graph state; all writes use upsert semantics	Idempotency
NFR-02	A single failed operation does not crash the pipeline; errors are caught per-item, logged, and skipped	Resilience
NFR-03	Every LLM call logs model, prompt tokens, completion tokens, and latency	Observability
NFR-04	Every node function is independently unit-testable with mocked dependencies	Testability
NFR-05	Deterministic outputs on all extraction, mapping, and grading nodes via zero temperature	Reproducibility
NFR-06	All thresholds and model names configurable via environment variables without code changes	Configurability
NFR-07	Processing a new or modified source file writes only the delta to the Knowledge Graph	Incremental

3.2 Architectural Overview

3.2.1 Two-Graph Architecture

The central architectural decision is the separation of the system into two independent LangGraph [29] state machines: a *Builder Graph* that constructs the

Knowledge Graph from raw inputs, and a *Query Graph* that answers natural-language questions over the constructed graph. This separation is inspired by the Command Query Responsibility Segregation (CQRS) principle: construction and interrogation have fundamentally different operational profiles—the Builder performs write-heavy, latency-tolerant batch processing, while the Query Graph demands read-heavy, latency-sensitive inference with strict groundedness guarantees.

Each graph is an independent LangGraph `StateGraph` with its own typed state schema (`BuilderState` and `QueryState`), its own set of nodes and conditional edges, and its own checkpointer for state persistence. The sole shared resource is the Neo4j Knowledge Graph, which the Builder writes to and the Query reads from.

The end-to-end data flow proceeds as follows. PDF documents (business glossaries, data dictionaries) and DDL schemas enter the Builder Graph, which extracts triplets, resolves entities, maps business concepts to physical tables, generates Cypher statements, and persists them to Neo4j. The Query Graph then retrieves context from the Knowledge Graph via hybrid retrieval, reranks it, generates an answer, and validates it through hallucination grading before returning a grounded response to the user.

3.2.2 Builder Graph Design

The Builder Graph is defined in `src/graph/builder_graph.py` and orchestrates the ontology construction pipeline through twelve nodes organised into six phases.

Phase 1: Document Ingestion. PDF documents are loaded via the OpenDataLoader PDF integration for LangChain and segmented using a two-level hierarchical chunking strategy. Parent chunks (800 tokens, 96-token overlap) provide rich context for downstream extraction; child chunks (256 tokens, 32-token overlap) provide fine granularity for vector similarity search. Both levels are embedded with BGE-M3 [19] and stored in Neo4j, connected by `CHILD_OF` edges.

Phase 2: Triplet Extraction and Entity Resolution. An LLM in JSON mode extracts structured (subject, predicate, object) facts from parent chunks, with a self-reflection loop that retries on malformed output (up to three attempts). A deterministic heuristic fallback using spaCy dependency parsing and regex pattern matching provides robustness when the LLM fails entirely. Extracted triplets then undergo entity resolution: BGE-M3 embeddings are used to compute a full cosine similarity matrix, Union-Find clustering groups candidate duplicates by similarity threshold, and an LLM judge adjudicates each cluster to produce canonical entities.

Phase 3: Schema Parsing and Enrichment. DDL files are parsed deterministically using sqlglot into typed schema objects (`TableSchema`, `ColumnSchema`). An optional LLM-based enrichment step expands cryptic abbreviations in column and table names (e.g., `CLI_NM` → "Client Name") to improve subsequent mapping

accuracy.

Phase 4: Semantic Mapping and Validation. For each physical table, a RAG-augmented mapping node retrieves the most relevant business entities from the Knowledge Graph and proposes a semantic alignment (`MappingProposal`). The proposal undergoes Actor-Critic validation: a Pydantic schema check verifies structural correctness, then an LLM Critic evaluates semantic correctness. If the Critic rejects the proposal, its critique is injected as a reflection prompt and the Actor retries, tracking the best-seen proposal across all attempts.

Phase 5: Cypher Generation and Healing. Validated mappings are converted into Cypher `MERGE` statements via few-shot prompting. Each Cypher statement is validated through an `EXPLAIN` dry-run against Neo4j. If a `CypherSyntaxError` occurs, the native error message is injected into a reflection prompt and the LLM retries generation (up to three attempts). On exhaustion, a deterministic parameterised Cypher builder produces a fallback statement immune to LLM quoting errors.

Phase 6: Graph Persistence. The validated Cypher is executed against Neo4j via batch `UNWIND` writes, which reduce the number of individual statements by approximately 87% compared to sequential `MERGE` operations. Foreign key edges (`REFERENCES`) are upserted as additional relationships between physical tables.

Table 3.2 summarises the Builder Graph nodes, their responsibilities, and their routing behaviour.

3.2.3 Query Graph Design

The Query Graph is defined in `src/generation/query_graph.py` and answers natural-language questions over the Knowledge Graph through nine nodes organised into five phases.

Phase 1: Hybrid Retrieval. Candidate context passages are gathered through three independent channels: (i) dense vector search using BGE-M3 embeddings on child chunks via the Neo4j vector index, (ii) sparse keyword matching using BM25 over chunk text via the rank-bm25 library, and (iii) graph traversal through Knowledge Graph relationships (seed-node expansion via `MENTIONS` and `MAPPED_TO` edges). The three result sets are fused using Reciprocal Rank Fusion (RRF) [41] with parameter $k = 60$, defined as:

$$\mathrm{RRF}(d) = \sum_{r \in R} \frac{1}{k + \mathrm{rank}_r(d)} \tag{3.1}$$

where R is the set of retrieval channels and $\mathrm{rank}_r(d)$ is the position of document d in the ranked list of channel r. RRF requires no weight tuning across channels, a practical advantage over linear combination alternatives.

Table 3.2: Builder Graph nodes and their responsibilities. Each node receives the full `BuilderState` and returns only the fields it modifies.

Node	Phase	Responsibility
`extract_triplets`	Extraction	Extract (subject, predicate, object) facts from chunks via LLM JSON mode or heuristic fallback
`entity_resolution`	Resolution	Block candidates via embedding similarity, cluster via Union-Find, adjudicate via LLM judge
`parse_ddl`	Schema	Deterministically parse DDL files into typed schema objects via sqlglot
`enrich_schema`	Schema	Expand cryptic abbreviations in column and table names via LLM
`parallel_mapping`	Mapping	Orchestrate concurrent mapping+validation for all tables via ThreadPool
`rag_mapping`	Mapping	Propose semantic alignment between a physical table and business entities via RAG retrieval
`validate_mapping`	Mapping	Two-phase validation: Pydantic structure check + LLM Critic semantic review (Actor-Critic)
`hitl`	Mapping	Human-in-the-loop: interrupt for manual review when confidence falls below threshold
`generate_cypher`	Cypher	Generate Neo4j Cypher MERGE statements from validated mappings via few-shot prompting
`heal_cypher`	Cypher	Self-correct Cypher syntax errors via reflection loop with EXPLAIN dry-run feedback
`build_graph`	Persistence	Execute validated Cypher against Neo4j via batch UNWIND writes
`save_trace`	Terminal	Routing node for pipeline completion and debug trace output

The retrieval parameters are configurable: vector search returns up to `retrieval_vector_top_` chunks (default 20), BM25 returns up to `retrieval_bm25_top_k` (default 10), and graph traversal is bounded by `retrieval_graph_traversal_limit` (default 30). Diversity in the merged result set is ensured by the RRF fusion, which naturally favours chunks that appear across multiple channels.

Phase 2: Reranking and Quality Gate. The fused candidates are rescored by a cross-encoder (bge-reranker-v2-m3) that jointly encodes each (query, chunk) pair for higher precision than bi-encoder scores alone. A retrieval quality gate then evaluates the sufficiency of the reranked context and assigns one of three outcomes: *proceed* (sufficient context for generation), *warning* (limited context, proceed with calibrated behaviour), or *abstain* (insufficient context, skip generation entirely).

Phase 3: Answer Generation. The answer generator receives the reranked context and the user query, and produces a response using few-shot prompting at temperature $T = 0.3$ for natural-language fluency. The system prompt is selected based on the context sufficiency level assigned by the quality gate, calibrating the LLM's behaviour to the available evidence.

Phase 4: Hallucination Grading. Inspired by the Self-RAG paradigm [10], a dedicated grader evaluates whether the generated answer is fully grounded in the retrieved context. The grader emits a structured decision with three fields: `grounded` (boolean), `critique` (textual explanation), and `action` (`pass` or `regenerate`). If the action is `regenerate`, the critique is injected as additional context and the answer generator retries. After a configurable maximum number of retries (default 3), the current answer is force-passed regardless of the grader's verdict. A consistency validator additionally checks for contradictory signals between the grader's boolean and action fields.

Phase 5: Finalisation. The finalised answer is assembled with metadata including source references, retrieved contexts, and quality metrics.

Table 3.3 summarises the Query Graph nodes.

3.2.4 Knowledge Graph Data Model

The Knowledge Graph is stored in Neo4j 5.x and follows a purpose-designed ontology with six node labels and five relationship types. Table 3.4 describes the node labels and their properties.

The five relationship types encode the semantic structure:

- `MAPPED_TO` (`BusinessConcept` → `PhysicalTable`): the core semantic mapping, carrying `confidence`, `validated_by`, and `created_at` attributes.
- `REFERENCES` (`PhysicalTable` → `PhysicalTable`): foreign key relationships with `column` (the foreign key column) and `references_column` (the referenced column) attributes.

Table 3.3: Query Graph nodes and their responsibilities.

Node	Phase	Responsibility
`hybrid_retrieval`	Retrieval	Three-channel retrieval (dense vector, BM25, graph traversal) with RRF fusion
`reranking`	Reranking	Cross-encoder rescoring of fused candidates
`retrieval_quality_gate`	Reranking	Evaluate context sufficiency; route to generation or early abstention
`context_distillation`	Preparation	Compress and filter reranked context for generation input
`answer_generation`	Generation	Generate response via few-shot LLM at $T = 0.3$; supports critique injection on retry
`hallucination_grader`	Validation	Self-RAG groundedness check; emit pass/regenerate decision
`grader_consistency_validator`	Validation	Verify internal consistency of the grader decision
`finalise`	Output	Assemble final answer with sources and metadata
`save_query_trace`	Terminal	Routing node for pipeline completion and debug trace output

Table 3.4: Knowledge Graph node labels and their properties.

Node Label	Properties	Purpose
`BusinessConcept`	`name` (unique), `definition`, `source_doc`, `synonyms`, `confidence_score`, `provenance_text`	Canonical business terms extracted from documentation
`PhysicalTable`	`table_name` (unique), `schema_name`, `column_names`, `column_types`, `ddl_source`	Tables parsed from DDL schemas
`Chunk`	`id` (unique), `text`, `embedding` (1024-dim vector)	Child chunks (256 tokens) for vector similarity search
`ParentChunk`	`id` (unique), `text`	Parent chunks (800 tokens) for extraction context
`SourceFile`	`path` (unique), `sha256_hash`, `ingestion_timestamp`	Tracks ingested files for incremental updates
`Attribute`	`name` (unique), `table_name`, `data_type`, `embedding` (1024-dim vector)	Column-level nodes for fine-grained retrieval

- `MENTIONS` (`Chunk` → `BusinessConcept`): links chunks to the business concepts they reference, enabling graph traversal retrieval.
- `CHILD_OF` (`Chunk` → `ParentChunk`): connects fine-grained child chunks to their richer parent chunks, enabling the Small-to-Big retrieval pattern where vector search matches child chunks and the system traverses to the parent for generation context.
- `HAS_COLUMN` (`PhysicalTable` → `Attribute`): links each physical table to its column-level `Attribute` nodes, enabling fine-grained retrieval at the column level.

Vector indexes are configured on `Chunk.embedding`, `BusinessConcept.embedding`, and `Attribute.embedding` using BGE-M3 1024-dimensional cosine similarity, supporting K-nearest-neighbour search for both retrieval and entity resolution blocking. Unique constraints on `BusinessConcept.name`, `PhysicalTable.table_name`, `Attribute.name`, and `SourceFile.path` enforce data integrity and ensure idempotent upserts via `MERGE`.

3.3 Core Design Patterns

3.3.1 Self-Reflection Loops

The system implements three distinct self-reflection loops, each tailored to a specific failure mode. This design draws on the Self-Refine [11] paradigm discussed in Section 2.4.3, adapting it to three different contexts within the pipeline.

Actor-Critic Validation (Mapping Phase). The Actor-Critic loop addresses the semantic correctness of proposed mappings. When the Actor (mapping node) produces a `MappingProposal`, it undergoes two validation layers: first, a Pydantic schema check verifies structural well-formedness; second, an LLM Critic evaluates whether the proposed concept-table alignment is semantically sound given the available entity context. If the Critic rejects the proposal, its textual critique is injected into a reflection prompt and the Actor regenerates. Critically, the state carries a `best_proposal` field that tracks the structurally valid proposal with the highest confidence score seen across all retry attempts. On critic exhaustion (maximum three retries), the system accepts the best proposal rather than the last rejected one, ensuring that transient failures in later attempts do not discard better proposals from earlier iterations.

Cypher Healing (Persistence Phase). The Cypher Healing loop addresses the syntactic correctness of generated Cypher statements. Each LLM-generated Cypher statement is validated through an `EXPLAIN` dry-run against Neo4j. If a `CypherSyntaxError` occurs, the native Neo4j error message is injected into a

reflection prompt that asks the LLM to fix the specific syntax issue. After a maximum of three healing attempts, the system falls back to a deterministic parameterised Cypher builder that produces a correct statement without LLM involvement, guaranteeing that persistence always succeeds.

Hallucination Grading (Query Phase). The hallucination grading loop addresses answer groundedness, following the Self-RAG paradigm [10]. The grader evaluates whether each claim in the generated answer can be traced to the retrieved context. If not, it emits a structured critique and a `regenerate` action, causing the answer generator to retry with the critique injected as additional context. A consistency validator checks for internal contradictions in the grader's output (e.g., `grounded=True` but `action=regenerate`). After a maximum of three retries, the answer is force-passed to prevent infinite loops.

All three loops share a common structure: generate → validate → inject feedback → retry, bounded by a configurable maximum attempt count. This bounded self-reflection pattern provides robustness without risking unbounded computation.

3.3.2 Confidence-Based Routing

Confidence-based routing is a cross-cutting mechanism that governs the system's behaviour at multiple decision points, trading off between thoroughness and efficiency.

Mapping confidence gate. When the Actor produces a `MappingProposal` with confidence ≥ 0.85, the Critic LLM call is skipped entirely—the proposal proceeds directly to approval. This optimisation reduces Critic LLM calls by approximately 60–70% while increasing the error rate for high-confidence mappings by only approximately 2%. For proposals with confidence below 0.85, the full Actor-Critic validation cycle runs. Proposals that pass validation but fall below the human-in-the-loop threshold (default 0.90) trigger a LangGraph `interrupt()`, pausing execution until a human reviews and approves, corrects, or rejects the proposal.

Retrieval quality gate. In the Query Graph, the quality gate evaluates the sufficiency of the retrieved context before generation begins. Three outcomes are possible: *proceed* routes to answer generation with a full-evidence system prompt; *warning* routes to generation with a calibrated prompt that accounts for limited context; *abstain* skips generation entirely and returns a direct statement that the available knowledge is insufficient to answer the question. This prevents the system from generating speculative answers from inadequate context.

3.3.3 LLM Tier Architecture

Different pipeline stages have different computational requirements. Schema enrichment is a simple acronym expansion task, while Cypher generation demands complex reasoning over graph patterns. To match model capability to task complexity while optimising cost, the system employs a five-tier LLM factory architecture.

Table 3.5 presents the five tiers, their roles, and their temperature settings. The temperature strategy is uniform across tiers: deterministic output ($T = 0.0$) for all structured data extraction and validation tasks, with fluency temperature ($T = 0.3$) reserved exclusively for answer generation, where natural language quality matters.

Table 3.5: LLM tier architecture: five tiers matched to task complexity.

Tier	Factory Function	Use Cases	Temp.
Lightweight	`get_lightweight_llm()`	Entity resolution judge, schema enrichment	0.0
Extraction	`get_extraction_llm()`	Triplet extraction in JSON mode	0.0
Mid-tier	`get_midtier_llm()`	RAG mapping, Actor-Critic validation, hallucination grading	0.0
Generation	`get_generation_llm()`	Answer generation	0.3
Reasoning	`get_reasoning_llm()`	Cypher generation and healing	0.0

Each tier is a singleton (`@lru_cache`) that returns an `InstrumentedLLM` wrapper providing automatic retry with exponential backoff, latency logging, and token usage tracking. All node functions type-annotate their LLM dependencies as `LLMProtocol`, a runtime-checkable structural type that decouples pipeline logic from specific LLM providers. The factory resolves providers through a three-level fallback chain: explicit per-tier configuration (preferred), global provider override (backward-compatible), and automatic provider detection from model name prefixes (legacy). This provider-agnostic design supports OpenRouter, OpenAI, Anthropic, Ollama, LM Studio, and other OpenAI-compatible endpoints without code changes.

3.3.4 Incremental Update Strategy

To satisfy NFR-07 (incremental updates), the system implements a SHA-256-based change detection mechanism. On each Builder run, every source file is hashed using streaming SHA-256 computation (64 KiB chunks, never loading the entire file into memory). The computed hash is compared against the stored hash in the corresponding `SourceFile` node in Neo4j. Files are classified as `unchanged` (hash matches), `modified` (hash differs), or `new` (no `SourceFile` node exists).

Unchanged files are skipped entirely. For modified files, a cascading delete removes all associated data—chunks, triplets, mappings, and the `SourceFile` node itself—followed by orphan cleanup that removes any `BusinessConcept` nodes with no remaining references. The file is then reprocessed from scratch and the new data committed via `MERGE` upserts. This strategy ensures that the Knowledge Graph is never rebuilt from scratch when only a subset of sources change.

3.4 Design Decisions and Trade-offs

Table 3.6 summarises the key design decisions, the rationale behind each, and the associated trade-off.

Table 3.6: Key design decisions and their trade-offs.

Decision	**Rationale**	**Trade-off**
Two-graph separation (Builder / Query)	Construction and interrogation have fundamentally different latency, consistency, and scaling requirements	Two state machines to maintain; shared Neo4j schema must be stable
Hierarchical chunking (800 / 256 tokens)	Parent chunks provide rich extraction context; child chunks enable precise vector search	Double text storage per document; only child chunks carry embeddings
Three-channel hybrid retrieval	No single channel covers all query types: vector captures semantics, BM25 captures identifiers, graph captures relationships	Increased retrieval complexity; requires fusion strategy (RRF)
Bounded self-reflection (max 3 retries)	Provides robustness without risking unbounded computation	May accept suboptimal results when three retries are insufficient
Confidence-based Critic bypass (≥ 0.85)	Reduces LLM calls by 60–70% on high-confidence mappings	Approximately 2% of bypassed mappings may contain errors
Deterministic Cypher fallback	Guarantees persistence always succeeds, even when LLM healing fails	Fallback Cypher is less optimised than LLM-generated alternatives
$T = 0.0$ for all structured tasks	Ensures deterministic, reproducible outputs for extraction, mapping, and grading	May miss creative solutions available at higher temperatures

3.5 Summary

This chapter presented the solution design of SemanticMesh. The system addresses the semantic mapping problem through a two-graph architecture that separates Knowledge Graph construction (Builder Graph, twelve nodes) from query answering (Query Graph, nine nodes), sharing a Neo4j data store with a purpose-designed ontology of six node labels and five relationship types. Three bounded self-reflection loops (Actor-Critic, Cypher Healing, Hallucination Grading) provide robustness against LLM errors at the mapping, persistence, and generation stages respectively. Confidence-based routing optimises LLM usage by skipping expensive validation when confidence is high, while a five-tier LLM factory matches model capability to task complexity. Incremental updates via SHA-256 change detection ensure that the Knowledge Graph is maintained without full reconstruction.

The following chapter details the implementation of these design decisions, covering the LangGraph orchestration, data models, prompt engineering strategies, and individual pipeline components.

Bibliography

[1] European Parliament and Council of the European Union. *Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation)*. 2016. URL: https://data.europa.eu/eli/reg/2016/679/oj (cit. on pp. 1, 21).

[2] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. *Attention Is All You Need*. 2017. arXiv: 1706.03762 [cs.CL]. URL: https://arxiv.org/abs/1706.03762 (cit. on pp. 1, 6).

[3] Tom Brown et al. *Language Models are Few-Shot Learners*. 2020. arXiv: 2005.14165 [cs.CL]. URL: https://arxiv.org/abs/2005.14165 (cit. on pp. 1, 7).

[4] Yunfan Gao et al. *Retrieval-Augmented Generation for Large Language Models: A Survey*. 2024. arXiv: 2312.10997 [cs.CL]. URL: https://arxiv.org/abs/2312.10997 (cit. on pp. 1, 8).

[5] Darren Edge et al. *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*. 2025. arXiv: 2404.16130 [cs.CL]. URL: https://arxiv.org/abs/2404.16130 (cit. on pp. 1, 2, 12, 13).

[6] Haoyu Han et al. *Retrieval-Augmented Generation with Graphs (GraphRAG)*. 2025. arXiv: 2501.00309 [cs.IR]. URL: https://arxiv.org/abs/2501.00309 (cit. on pp. 2, 12).

[7] Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. *Graph Retrieval-Augmented Generation: A Survey*. 2024. arXiv: 2408.08921 [cs.AI]. URL: https://arxiv.org/abs/2408.08921 (cit. on pp. 2, 12).

[8] Yuntong Hu, Zhihan Lei, Zheng Zhang, Bo Pan, Chen Ling, and Liang Zhao. *GRAG: Graph Retrieval-Augmented Generation*. 2025. arXiv: 2405.16506 [cs.LG]. URL: https://arxiv.org/abs/2405.16506 (cit. on pp. 2, 12, 13).

[9] Mayank Kejriwal. *Named Entity Resolution in Personal Knowledge Graphs*. 2023. arXiv: 2307.12173 [cs.AI]. URL: https://arxiv.org/abs/2307.12173 (cit. on pp. 2, 13).

[10] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. *Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection*. 2023. arXiv: 2310.11511 [cs.CL]. URL: https://arxiv.org/abs/2310.11511 (cit. on pp. 2, 3, 10, 27, 30).

[11] Aman Madaan et al. *Self-Refine: Iterative Refinement with Self-Feedback*. 2023. arXiv: 2303.17651 [cs.CL]. URL: https://arxiv.org/abs/2303.17651 (cit. on pp. 3, 18, 29).

[12] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. *Ragas: Automated Evaluation of Retrieval Augmented Generation*. 2025. arXiv: 2309.15217 [cs.CL]. URL: https://arxiv.org/abs/2309.15217 (cit. on pp. 3, 19).

[13] Lianmin Zheng et al. *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. 2023. arXiv: 2306.05685 [cs.CL]. URL: https://arxiv.org/abs/2306.05685 (cit. on pp. 4, 18).

[14] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg Corrado, and Jeffrey Dean. *Distributed Representations of Words and Phrases and their Compositionality*. 2013. arXiv: 1310.4546 [cs.CL]. URL: https://arxiv.org/abs/1310.4546 (cit. on p. 5).

[15] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. *GloVe: Global Vectors for Word Representation*. 2014. URL: https://nlp.stanford.edu/pubs/glove.pdf (cit. on p. 5).

[16] Matthew E. Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. *Deep Contextualized Word Representations*. 2018. arXiv: 1802.05365 [cs.CL]. URL: https://arxiv.org/abs/1802.05365 (cit. on p. 5).

[17] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. 2019. arXiv: 1810.04805 [cs.CL]. URL: https://arxiv.org/abs/1810.04805 (cit. on pp. 5, 7).

[18] Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. *Multilingual E5 Text Embeddings: A Technical Report*. 2024. arXiv: 2402.05672 [cs.CL]. URL: https://arxiv.org/abs/2402.05672 (cit. on p. 6).

[19] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. *BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation*. 2024. arXiv: 2402.03216 [cs.CL]. URL: https://arxiv.org/abs/2402.03216 (cit. on pp. 6, 24).

[20] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. 2023. arXiv: 2201.11903 [cs.CL]. URL: https://arxiv.org/abs/2201.11903 (cit. on pp. 7, 17).

[21] Long Ouyang et al. *Training language models to follow instructions with human feedback*. 2022. arXiv: 2203.02155 [cs.CL]. URL: https://arxiv.org/abs/2203.02155 (cit. on p. 7).

[22] Hugo Touvron et al. *LLaMA: Open and Efficient Foundation Language Models*. 2023. arXiv: 2302.13971 [cs.CL]. URL: https://arxiv.org/abs/2302.13971 (cit. on p. 7).

[23] Albert Q. Jiang et al. *Mistral 7B*. 2023. arXiv: 2310.06825 [cs.CL]. URL: https://arxiv.org/abs/2310.06825 (cit. on p. 7).

[24] Hai Toan Nguyen, Tien Dat Nguyen, and Viet Ha Nguyen. *Enhancing Retrieval Augmented Generation with Hierarchical Text Segmentation Chunking*. 2025. arXiv: 2507.09935 [cs.CL]. URL: https://arxiv.org/abs/2507.09935 (cit. on p. 8).

[25] Muhammad Arslan Shaukat, Muntasir Adnan, and Carlos C. N. Kuhn. *A Systematic Investigation of Document Chunking Strategies and Embedding Sensitivity*. 2026. arXiv: 2603.06976 [cs.CL]. URL: https://arxiv.org/abs/2603.06976 (cit. on p. 8).

[26] Hervé Déjean, Stéphane Clinchant, and Thibault Formal. *A Thorough Comparison of Cross-Encoders and LLMs for Reranking SPLADE*. 2024. arXiv: 2403.10407 [cs.IR]. URL: https://arxiv.org/abs/2403.10407 (cit. on p. 9).

[27] Tong Chen, Hongwei Wang, Sihao Chen, Wenhao Yu, Kaixin Ma, Xinran Zhao, Hongming Zhang, and Dong Yu. *Dense X Retrieval: What Retrieval Granularity Should We Use?* 2024. arXiv: 2312.06648 [cs.CL]. URL: https://arxiv.org/abs/2312.06648 (cit. on p. 9).

[28] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. *Corrective Retrieval Augmented Generation*. 2024. arXiv: 2401.15884 [cs.CL]. URL: https://arxiv.org/abs/2401.15884 (cit. on p. 9).

[29] Aditi Singh, Abul Ehtesham, Saket Kumar, Tala Talaei Khoei, and Athanasios V. Vasilakos. *Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG*. 2026. arXiv: 2501.09136 [cs.AI]. URL: https://arxiv.org/abs/2501.09136 (cit. on pp. 10, 23).

[30] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. *ReAct: Synergizing Reasoning and Acting in Language Models*. 2023. arXiv: 2210.03629 [cs.CL]. URL: https://arxiv.org/abs/2210.03629 (cit. on pp. 11, 17).

[31] Lei Wang et al. *A Survey on Large Language Model based Autonomous Agents*. 2025. DOI: https://doi.org/10.1007/s11704-024-40231-1. arXiv: 2308.11432 [cs.AI]. URL: https://arxiv.org/abs/2308.11432 (cit. on p. 11).

[32] Milena Trajanoska, Riste Stojanov, and Dimitar Trajanov. *Enhancing Knowledge Graph Construction Using Large Language Models*. 2023. arXiv: 2305.04676 [cs.CL]. URL: https://arxiv.org/abs/2305.04676 (cit. on p. 12).

[33] Jiacheng Liang, Yuhui Wang, Changjiang Li, Rongyi Zhu, Tanqiu Jiang, Neil Gong, and Ting Wang. *GraphRAG under Fire*. 2025. arXiv: 2501.14050 [cs.LG]. URL: https://arxiv.org/abs/2501.14050 (cit. on p. 13).

[34] Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Xu, Liu Ziyi, Yue Yu, Wei-Ying Ma, and Chao Zhang. *LightRAG: Simple and Fast Retrieval-Augmented Generation*. 2024. arXiv: 2410.05779 [cs.IR]. URL: https://arxiv.org/abs/2410.05779 (cit. on p. 13).

[35] Armin Oliya, Amir Saffari, Priyanka Sen, and Tom Ayoola. *End-to-End Entity Resolution and Question Answering Using Differentiable Knowledge Graphs*. 2021. arXiv: 2109.05817 [cs.CL]. URL: https://arxiv.org/abs/2109.05817 (cit. on p. 15).

[36] Sander Schulhoff et al. *The Prompt Report: A Systematic Survey of Prompt Engineering Techniques*. 2025. arXiv: 2406.06608 [cs.CL]. URL: https://arxiv.org/abs/2406.06608 (cit. on p. 15).

[37] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. *Large Language Models are Zero-Shot Reasoners*. 2022. arXiv: 2205.11916 [cs.CL]. URL: https://arxiv.org/abs/2205.11916 (cit. on p. 17).

[38] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. *Reflexion: Language Agents with Verbal Reinforcement Learning*. 2023. arXiv: 2303.11366 [cs.AI]. URL: https://arxiv.org/abs/2303.11366 (cit. on p. 18).

[39] Aman Tiwari, Shiva Krishna Reddy Malay, Vikas Yadav, Masoud Hashemi, and Sathwik Tejaswi Madhusudhan. *Auto-Cypher: Improving LLMs on Cypher generation via LLM-supervised generation-verification framework*. 2025. arXiv: 2412.12612 [cs.CL]. URL: https://arxiv.org/abs/2412.12612 (cit. on p. 18).

[40] Ziije Zhong, Linqing Zhong, Zhaoze Sun, Qingyun Jin, Zengchang Qin, and Xiaofan Zhang. *SyntheT2C: Generating Synthetic Data for Fine-Tuning Large Language Models on the Text2Cypher Task*. 2025. arXiv: 2406.10710 [cs.AI]. URL: https://arxiv.org/abs/2406.10710 (cit. on p. 18).

[41] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. «Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods». In: *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2009, pp. 758–759. DOI: 10.1145/1571941.1572114 (cit. on p. 25).